P03 Context-Aware Vulnerability Prioritization Engine

Domain	Defensive Architecture
Team Size	2 students (Myunghee Jung & Costantinos Boyiatzis)
Duration	2026-09-10 ~ 2026-12-03
Faculty Advisor	Mohammad Amr Khan

Version	Date	Author	Changes
V1.0	2026-09-22	Costantinos Boyiatzis	Initial draft – section 1-3
V2.0	2026-09-23	Myunghee Jung	Added – section 4-15 and appendix
V3.0	2026-09-24	Myunghee Jung	Updated – section 4, 5, 6, 9, 12
V4.0	2026-10-26	Myunghee Jung	Added – section 4-1-2, 4-1-3 Updated – section 3, 5

1. Background & Problem Statement

Most organizations default to patching in raw CVSS order, treating a critical finding on a test box the same as an identical finding on an internet facing production database. Effective prioritization depends on context that CVSS alone does not capture: patch deploy risk, compensating controls and asset criticality. This project builds a scoring engine that combines all four dimensions, (CVSS, patch difficulty, compensating controls and asset criticality) and measures whether that re prioritization produces a better ordered list than a CVSS only baseline.

2. Objectives

a. Define a composite scoring model spanning CVSS patch difficulty, compensating controls and asset criticality.
b. Build an ingestion pipeline normalizing output and asset inventory into a common schema.
c. Implement the scoring engine with documented, tunable weights and a sensitivity analysis.
d. Validate the model's prioritization against a raw CVSS only baseline using structured expert comparison.

3. Methodology

Setup a small lab range with deliberately vulnerable, differently critical hosts and scan it with an open-source scanner. Build an asset inventory schema tagging each host with a criticality tier and a compensating-controls profile (drawn from the CIS Controls framework). Encode patch difficulty as effort/risk tiers. Combine all four dimensions into a composite score with documented weighting. Run a sensitivity analysis, then validate against the CVSS-only

baseline through a structured comparison, reporting rank correlation and a qualitative account of where and why the model disagrees with raw CVSS.

3-1. Data Collection & Baseline Inputs

The development of the prioritization model relies on foundational data provided by external experts. Specifically, a baseline list of vulnerabilities including existing controls and asset criticality was provided by the Advisor on October 14. This dataset serves as the primary input for the project team to evaluate and assign subsequent weight scores.

4. Scope

4-1. In Scope

4-1-1. Risk Scoring Factors

Dimension	Source	Scale	Weight
Severity	NVD API (CVSS/CVE)	0-10	40%
Exploited in the Wild	CISA KEV, Vendor Advisories, Threat Intelligence Feeds	0-2	15%
PoC / Exploit Availability	Exploit-DB, GitHub, Security Research Publications	0-2	10%
Patch Difficulty	Vendor Advisories	1-3	10%
Compensating Controls	Firewall Rules, Network Segmentation, EDR, IPS	0-3	10%
Asset Criticality	Asset Classification / Host Tagging	1-4	10%
Risk Acceptance / Transfer	Approved Risk Register, Cyber Insurance, Third-Party Contracts	0-2	5%

4-1-2. Risk Data Fields Mapping Logic

	Weight	Target Metadata Fields	Mapping Logic & Scoring Basis
Severity	40%	cvss_base_score cvss_severity	The baseline technical severity of the vulnerability. The cvss_base_score (0–10) directly determines this score.
Exploited in the Wild	15%	exploitation_context data_status	Active threat status. Points (0–2) are awarded if the CVE is listed on the CISA KEV or flagged in threat intel feeds.
PoC / Exploit Availability	10%	exploitation_context	Public availability of exploit code. Points (0–2) are driven by

			lookups in Exploit-DB, GitHub, or security research.
Patch Difficulty	10%	patch_complexity estimated_remediation_effort recommended_remediation	Complexity of the fix. Higher points (1–3) are assigned if remediation requires service downtime, reboots, or complex configurations.
Compensating Controls	10%	compensating_controls	Existing mitigations (firewalls, EDR, IPS). Lower/No controls yield a higher score (0–3) as the asset is left exposed.
Asset Criticality	10%	asset_group, critical_assets internet_facing_assets exposure_classification	The value and exposure of the host. Points (1–4) increase if the asset is internet-facing or belongs to a core infrastructure group.
Risk Acceptance / Transfer	5%	data_status	Exception tracking. Indicates if the risk is formally accepted or transferred. Typically acts as a score reducer (0–2).

4-1-3. Exposure classification

Exposure Classification	Option A *(Traditional Security)*	Option B *(Business Impact-Centric)*	Option C *(Hybrid / Mitigating Controls)*
internet-facing	4 (Critical)	1 (Low)	4 (Critical)
internal	2 (Medium)	2 (Medium)	2 (Medium)
restricted / segmented	1 (Low)	4 (Critical)	3 (High)
Primary Focus	Attack Surface Exposure Direct perimeter visibility and ease of remote exploitation.	Business Continuity Operational fallout and security of core crown-jewel assets.	Balanced Strategy Weighs public ingress routes against structural firewall isolation.

4-1-4. Score Interpretation

Risk Score	Risk Level	Interpretation
1-6	Low	Low likelihood and limited business impact. Routine monitoring is sufficient.
7-12	Medium	Moderate risk requiring planned remediation and monitoring.
13-18	High	Significant risk with potential operational or security impact. Prioritized remediation is recommended.
19-24	Critical	Severe risk requiring immediate attention and remediation.

4-1-3. Scoring and Validation

- **Composite score:** A weighted combination of the 4 dimensions, with documented weights you can tune.
- **Sensitivity analysis:** Measures how much the ranking changes when you change the weights.
- **Validation:** Compares the ranking to a CVSS-only baseline, using rank correlation (e.g., Kendall's τ) and a written account of where and why the two disagree.

4-1-4. Environment and Tooling

- **Lab Environment**

Role	Machine	OS / Stack	Virtualization
Host (x2)	Each member's own PC	Windows 11/ Windows 11	
Attacker	Guest VM	Kali Linux	VMware Workstation Pro 26H1
Victim 1	Guest VM	Ubuntu 16.04	
Victim 2	Guest VM	Win Server 2016	
Victim 3	Guest VM	Win 7	
Victim 4	Guest	Ubuntu + DVWA (PHP+MySQL) web app	

- **Stack:** Python, Pandas, OpenVAS/Greenbone (or Nessus Essentials), NVD API, the scoring engine and a Flask/Streamlit dashboard.

4-2. Out of Scope

- No production systems are scanned.

4-3. Scope Adjustment

- The current scope assumes a three-person project team. A reduction to two resources may necessitate scope adjustment to align with available capacity.
- It shall be reviewed during the Formal Advisor Check-ins on October 8 and November 5.

5. Deliverables

- Weighted Vulnerability Prioritization List
- Asset Inventory and Classification Schema
- Composite Prioritization Model and Weighting Framework
- Sensitivity Analysis Results
- Comparative Evaluation against a CVSS-only baseline
- Vulnerability Scan Results from the lab environment
- Technical Report
- Recorded Demo

6. Budget & Resource Estimate

This is an academic project with no monetary budget line; the estimate below covers time and infrastructure resources the team must secure.

Resource	Estimate	Notes
Team effort	20 hrs/week per member × 2 members × 12 weeks	Heavier in W3-4, W8-9, W11-12
Lab infrastructure	3–5 VMs	No cost - hosted on personal hardware Budget for cloud credits if used
Software / licensing	$0 expected	OpenVAS/Greenbone Nessus Essentials (free tier) Python stack NVD API
Contingency	~10% schedule buffer	• True Deadline: 2026-12-03 • Internal Target Date: Nov 30 (3-day schedule buffer)

7. Decision-Making & Escalation Process

With a 2-person team, work is reviewed by the other member before being considered done. e.g., Member 1 reviews the scoring engine, Member 2 reviews the validation design.

Both members must agree

- Any change to the four core scoring dimensions, the project's objectives, or a listed deliverable.
- Any milestone date change, and any temporary reallocation of tasks to manage availability risk.

Escalate to Faculty Advisor

- The two members cannot reach agreement on a decision above.
- Any milestone date change of more than one week.
- Any request to scan systems outside the team's own isolated lab range.

Stakeholder communication and blocker escalation are shared responsibilities; either member may represent the team, and significant communications are shared with both before being sent.

8. Team Agreement

- Both team members serve as each other's backup to ensure continuity and shared ownership of project deliverables.
- Myunghee leads the scanning and tooling stack (Vulnerable Services VM, OpenVAS/Greenbone, Nessus Essentials, and NVD API) while providing knowledge transfer and hands-on guidance to Costantinos to ensure shared understanding and operational continuity.
- Costantinos leads the scoring engine and Flask/Streamlit dashboard development while providing knowledge transfer and walkthroughs of the design and implementation to ensure shared understanding and operational continuity.

- Each team member will complete their assigned documentation updates weekly to maintain steady progress and avoid last-minute work on the final technical report.
- Team members will regularly share new technical skills, tools, and lessons learned throughout the project to promote cross-training and collective growth.
- When a team member encounters technical challenges or blockers, the other member will provide support through troubleshooting, peer review, brainstorming, and collaborative problem-solving.
- Key technical decisions, implementation details, and setup procedures will be documented to facilitate knowledge sharing and reduce single-person dependencies.

9. Methodology & Tooling

Stand up a small lab range and scan it with an open-source scanner (OpenVAS/Greenbone or Nessus Essentials). Build an asset inventory schema tagging each host with criticality tier and compensating-controls profile. Encode patch difficulty as effort/risk tiers. Combine all four dimensions into a composite score with documented weighting, run sensitivity analysis, then validate against the CVSS-only baseline through a structured comparison.

Category	Details
Stack	• Python; OpenVAS/Greenbone or Nessus Essentials • NVD API for CVSS/CVE data • Lightweight scoring engine • Flask/Streamlit dashboard
Data	• Scanner findings from the team's own lab range • Synthetic asset-criticality and compensating-control metadata built against the CIS Controls framework • *List of vulnerabilities with controls and criticality by advisor*
Lab	• Small VM range (3–5 hosts) with intentionally vulnerable services, tagged with varied criticality • No production systems scanned • Victim VMs will be based on the Ubuntu 16.04, Windows 7, and Windows Server 2016 environments. The target will include DVWA (PHP + MySQL) and/or systems with outdated packages. However, these VMs may change during the engine development and validation phases, if necessary.

10. Milestone Schedule (13 Weeks)

Weeks	Milestone	Output / Deliverable	Phase
1–2	Team Transition (3 → 2 members) & Re-planning	• Initial stakeholder alignment Jira onboarding and workflow setup • Project Charter development • Risk Register creation	Initiation
3–4	Model dimensions scoped; lab range and scanner stood up; asset criticality and compensating-control taxonomies drafted	Scoping doc; scanning pipeline; taxonomies v1	Initiation
3–4	Vulnerability ingestion pipeline built; CVSS-only baseline prioritized list produced as the comparison point	Ingestion pipeline; baseline ranked list v0	Initiation
5–7	Patch-difficulty and compensating-control models implemented; composite scoring engine with documented tunable weights	Scoring engine v1; prioritized list v1	Execution
8–9	Sensitivity analysis on weighting scheme; structured validation (expert or scenario-panel comparison) designed and run	Sensitivity analysis; validation results	Execution
10–11	Scoring model refined; dashboard built for SOC-style consumption of the prioritized list	Scoring engine v2; dashboard	Validation
12	Packaging, documentation, and a final full-range run for the report	Tool repo with docs	Validation
13	Final report and demo to advisor	Final report; recorded demo	Close

11. Team Roles & Responsibilities

RACI Matrix

Task	Myunghee (PM Lead, Scanning Lead)	Costantinos (Scoring Model Lead)	Advisor
Define Scope	R	R	A
Run Vulnerability Scan / Ingestion	R	C	I
Build Scoring Engine	C	R	I
Sensitivity Analysis & Validation Study	R	R	I
Write Final Report	R	R	I
Final Demo	R	R	A

R = Responsible (does the work) A = Accountable (owns outcome, signs off) C = Consulted (provides input) I = Informed (kept up to date)

12. Team Communication & Availability

12-1. Communication Plan

Stakeholder	Information Need	Channel	Frequency	Owner
Course Instructor / Advisor	Progress, milestones, risks, and key decisions	MS-Teams CourseLink/ Email MS-Teams Messenger	Weekly As needed	All
Team	Task status, blockers, action items	WhatsApp (1:1 messaging) Monday 2 PM – 2:30 PM	Daily	All
Team	Stand-up updates (What was completed, next steps, blockers)	MS-Teams	Weekly (30 min)	MJ, CB
Team	Decision log, minutes	Shared Doc (Jira)	Per meeting	PM Lead

- Project communications with advisor will occur on a weekly basis. A brief status meeting will be held every Thursday to review ongoing work, progress made, and any issues or risks identified. The meeting will also provide an opportunity for stakeholders and team members to raise questions, discuss concerns, and clarify project-related matters.
- Team members will maintain daily communication.
- Major updates, blockers, and decisions will be communicated in a timely manner.
- A 30-minute weekly stand-up will be held to discuss:
  - ✓ What was completed
  - ✓ What will be completed next
  - ✓ Any blockers or support needed
- All significant decisions and supporting rationale will be documented in the decision log.
- Meeting minutes will be distributed within 24 hours when applicable.
- Escalation path: Team → Instructor/Advisor

12-2. Availability Constraints

Team Member	Availability Constraint
Myunghee	Unavailable on Tuesdays from 12:00 PM to 5:00 PM due to class commitments.
Costantinos	Unavailable on Wednesdays from 12:00 PM to 2:00 PM.

13. Risk Register (Initial)

Risk ID	Risk Title	Risk Description	Likelihood	Impact	Mitigation Strategy	Contingency Plan	Owner
R1	**Team Member Sickness or Emergency**	**If one team member becomes sick, faces an emergency, or is unable to work, approximately**	**Medium**	**High**	**- Daily progress sync and knowledge sharing through a shared repository.**	**- Remaining member temporarily assumes critical responsibilities using shared documentation**	**Team**

		50% of project progress may stop due to the two-person team structure.			**- Keep documentation, code, and project files updated daily. - Cross-train on critical project tasks and environments.**	**and repository resources. - Discuss with advisor remaining scope adjustment.**	
R2	**Insufficient Findings from Small Lab Environment**	**The limited number of vulnerable systems in the lab may not generate enough scan results or meaningful findings for analysis and reporting.**	**Medium**	**Medium**	**- Include multiple vulnerability sources (Metasploitable, DVWA, intentionally vulnerable services). - Run different scan configurations and validation tests. - Collect findings continuously throughout development.**	**- Supplement findings with documented case studies, archived scan results, or additional vulnerable hosts.**	**Ingestion / Scanning Lead**
R3	**Write-up and Reporting Deliverable Deprioritized**	**Development and testing activities may consume most project time, causing delays in final documentation and report preparation.**	**Medium**	**High**	**- Allocate dedicated weekly documentation sessions. - Update report sections incrementally throughout the project. - Maintain report templates and evidence as activities occur.**	**- Shift focus to documentation during final project phase and prioritize required report sections.**	**Team**
R4	**VM Lab Range Instability**	**Vulnerable VMs may crash, lose state, or consume excessive host resources, reducing available scan targets and testing capability.**	**High**	**High**	**- Create snapshots immediately after VM configuration. - Document exact VM specifications and setup procedures. - Monitor resource utilization regularly.**	**- Restore affected VMs from snapshots. - Reduce lab to a minimum of three functional hosts. - Use pre-recorded scan XML results if VM uptime cannot be maintained.**	**Team**
R5	**Scope Creep / Unrealistic Project Scope**	**Additional features and enhancements may exceed what a two-person**	**High**	**High**	**- Prioritize features using Jira (Must-Have vs Nice-to-Have).**	**- Reduce project scope to MVP and defer non-essential functionality.**	**Team**

		team can realistically complete within the project timeline.			**- Define a realistic MVP. - Conduct weekly scope reviews and remove low-priority items when necessary.**		
R6	**Recorded Demo Submission Delay Due to Last-Minute Technical Bugs**	**Unexpected software defects, crashes, or integration issues close to submission may delay recording or reduce demo quality.**	**Medium**	**High**	**- Freeze all development W11 EOD - Complete testing before the freeze date. - Record an early draft demo once core functionality is available.**	**- Use the pre-recorded backup demo and apply only critical fixes after freeze.**	**Team**
R7	**Schema Incompatibility Between Scanner Formats**	**Different vulnerability scanners may produce XML or report outputs with inconsistent field structures, causing import and parsing failures.**	**Medium**	**Medium**	**- Define and document a standardized internal schema early. - Test parser compatibility with sample outputs from each scanner. - Implement schema validation checks.**	**Create scanner-specific transformation scripts or limit support to validated formats for the final deliverable.**	**Team**

14. Ethics & Authorization Boundaries

All scanning is confined to the team's own isolated lab range. Asset-criticality and compensating-control metadata are synthetic or self-generated for the lab range.

15. Success Criteria

 Scoring engine produces a documented, tunable composite ranking distinct from raw CVSS ordering.
 Sensitivity analysis clearly shows how ranking changes as weights vary.
 Validation study reports a rank correlation statistic, and a documented, qualitative account of where and why the model diverges from CVSS-only prioritization.
 Draft Technical Report is delivered on schedule (Nov 26).
 Presentation slides, demo, and final documentation completed (Dec 03).
 Final Technical Report, Source Code, and Recorded Demo are submitted on schedule (Dec 03).

Appendix A. Scope Change Log

Version	Date	Change	Reason
1.0	W1	Initial Scope	
1.1	W5		Due to reduced team capacity (3 → 2 members)
1.2	W9		